

Types de données

Toute valeur de donnée appartient à un **type**

```
>>> type(25)
```

```
<class 'int'>
```

Les **entiers**

```
>>> type("Bonjour")
```

```
<class 'str'>
```

Les **strings**

(chaînes de caractères)

```
>>> type(37.2)
```

```
<class 'float'>
```

Les **réels**

(réels en virgule flottante)

```
>>> type(True)
```

```
<class 'bool'>
```

Les **booléens**

Type de données

Un type de données =

- un **ensemble de valeurs** appartenant au type : le **domaine**
- un **ensemble d'opérations** applicables aux valeurs du type

`type(x)` retourne (une valeur représentant) le type de `x`

```
>>> x=5
```

```
>>> type(x)
```

```
<class 'int'>
```

```
>>> type(type(x))
```

```
<class 'type'>
```

```
>>> type(type(type(x)))
```

```
<class 'type'>
```

Entiers

- Valeurs 0, 1, 2, ..., -1, -2, ...

- Illimités :

10 ** 1000

mille chiffres

1000...000

- Opérations

x = 2

priorité :

**

*, /, %

+, -

3 * x ** 2 + 4 * x - 2 18

(42 // 5) * 5 + 42 % 5 42

division entière

42 / 5 = 8.4

Réels

- Valeurs 0.0, 1.0, 0.01, 3e8, 6.02e-23, ...

3x10⁸

6.02x10⁻²³

- Représentés en "virgule flottante"

mantisse x 10^{exposant}

nombre très grands ou très petits (exposant),
à précision constante (mantisse)

- Opérations

x = 0.5

3.0 * x ** 2 + 4.0 * x - 2.0 0.75

(42 / 3) * 3 42.0

division réelle

Chaînes de caractères

- Valeurs "Bonjour", 'Salut',
"Il dit 'non'",
"" "Sur
plusieurs
lignes""

- Opérations

"2" + "2"

"22"

"infor" + "matique"

"informatique"

concaténation
PAS addition !

Booléens

- Deux valeurs `True`, `False`
- Opérations

`p = True`

`q = False`

<code>p and q</code>	<code>False</code>
----------------------	--------------------

<code>p or q</code>	<code>True</code>
---------------------	-------------------

<code>not p</code>	<code>False</code>
--------------------	--------------------

<code>not (p and not q)</code>	<code>False</code>
--------------------------------	--------------------

Opérations booléennes

- Négation logique (**not**)

a	!a
True	! True = False
False	! False = True

- ET logique (**and**)

a/b	True	False
True	True and True = True	True and False = False
False	False and True = False	False and False = False

- OU logique (**or**)

a/b	True	False
True	True or True = True	True or False = True
False	False or True = True	False or False = False

Egalité et comparaison

Opérandes quelconques, résultat booléen

- Egalité de valeurs **==**

possible entre toute paire de valeurs

5 == 3+2 True

5 == 6 False

5 == 5.0 True

5 == "5" False

"toto" == "to"+"to" True

- Comparaisons de valeurs **!=, <, >, <=, >=**

Affectation abrégée

Opération et affectation en même temps

x += 1 "Ajouter 1 à x"

Equivalait à **x = x + 1**

Aussi pour **-**, *****, ******, **/**, **//**, **%**

x = 10 ; x -= 2

x = 8

x = 10 ; x *= 5

x = 50

x = 10 ; x //= 3

x = 3

x = "10" ; x += "10"

x = "1010"

(mais PAS de **x++**)

Conversion de types

`int(x)` convertit `x` en valeur **entière**

`int(3.14)` 3

`int(3.999)` 3 (arrondi vers 0)

`int("42")` 42

`int("15cm")` **ERREUR**

`float(x)` convertit `x` en valeur **réelle**

`float(42)` 42.0

`str(x)` convertit `x` en chaîne de caractères

`str(3.14)` "3.14"

Input

Entrer des données de l'utilisateur ?

```
>>> nom = input("Votre nom? ")  
Votre nom? Charles  
>>> nom  
'Charles'
```

input(msg)

- imprime msg à la console,
- lit un texte à la console et
- retourne ce texte.

Le résultat est un string!

```
age = int(input("Votre âge? "))  
lustres = age//5
```